

Fundamentos para el desarrollo de Agentes de IA

MSc. Jesus Gastañaduy

Agenda



1. Tool calling múltiple y skills

De varias tools sueltas a una skill que las orquesta



2. Modelos locales vs. modelos propietarios

Costos, privacidad y flexibilidad vs. vanguardia e infraestructura



3. Caso práctico: Peru Fire Analysis Skill

SKILL.md, tools.py y un Deep Agent corriendo en local

De múltiples tools a una skill



Tool calling
múltiple



¿Qué es una
skill?



¿Por qué usar
skills?



Concepto clave

¿Qué es una skill?

Si una tool es una operación básica, podemos pensar en una skill como la fórmula que detalla cómo combinar mis tools básicas.

- Vive en una carpeta con un archivo SKILL.md que describe cuándo y cómo usarla
- El agente la carga bajo demanda, no en cada turno de conversación

En DeepAgents

```
skills=[str(SCRIPT_DIR) ]
```

Apunta a la carpeta que contiene el SKILL.md

¿Por qué usamos skills?

Para optimizar costos y llamarlas solo cuando es estrictamente necesario



Menor costo

Una skill se carga bajo demanda, no en cada llamada al agente



Menos tokens

La alternativa —detallar todo en el system prompt— es ineficiente en tokens



Mejor calidad

Un system prompt sobrecargado afecta negativamente la calidad del output

¿Por qué usar modelos locales?

Correr el modelo en tu propio hardware, sin depender de un proveedor externo



Costos

Son totalmente gratis, sin importar el número de queries que hagas



Privacidad

Tu información nunca sale de tu ordenador



Flexibilidad

Posibilidad de hacer fine-tuning sobre el modelo

¿Por qué usar modelos propietarios?

La alternativa: pagar por acceso a los modelos más avanzados



Vanguardia

Los mejores modelos del momento suelen ser propietarios



Infraestructura

Correr queries a escala requiere grandes costos fijos



Costos

Puede ser más barato pagar por tokens que montar la arquitectura para correr modelos open source

Caso práctico: Peru Fire Analysis Skill



tools.py



SKILL.md



Demo en vivo
(local)



Pieza 1

tools.py

5 funciones de Python que comparten estado entre sí: se llaman en secuencia dentro de una misma sesión.

- `download_viirs_snpp_peru(year)` descarga los focos de calor VIIRS-SNPP de NASA FIRMS
- `fires_clean()` y `download_ctry_shp()` limpian los puntos y traen los límites de Perú (GADM)
- `intersect_and_collapse_fires_peru()` y `folium_choropleth_peru()` agregan por departamento y generan el mapa

Estado compartido

```
_SESSION_DATA = { }
```

Diccionario global que pasa datos de una tool a la siguiente



Pieza 2

SKILL.md

El archivo que el agente carga bajo demanda para saber cuándo y en qué orden usar las tools.

- El frontmatter (name, description) le indica al agente cuándo cargar la skill
- 'Tools Reference' documenta parámetros y retorno de cada función
- 'Workflow' fija el orden: download → clean → boundaries → intersect → map

Ejemplo (frontmatter)

```
---  
name: peru-fire-analysis  
description: Analyze and  
visualize NASA VIIRS  
wildfire data for Peru  
---
```

El agente decide solo, con esta descripción, cuándo usarla



En conjunto

Las 5 tools en secuencia

Cada tool depende de la anterior: comparten estado dentro de `_SESSION_DATA`.

Definidas en `tools.py` y expuestas al agente con `tools=[...]`

`download_viirs()`

`fires_clean()`

`download_shp()`

`intersect_fires()`

`choropleth()`

Cinco funciones encadenadas: cada una alimenta a la siguiente



SKILL.md las orquesta

El agente sigue el 'Workflow' del SKILL.md para llamarlas en el orden correcto.

Nuestro Geo Agente en pocas líneas

local_llm nos conecta con Nuestro modelo local; agent conecta todas las piezas

```
local_llm = ChatOpenAI(
    base_url="http://127.0.0.1:1234/v1",  # LM Studio
    api_key="lm-studio",
    model="google/gemma-4-12b-qat",
    temperature=0
)
agent = create_deep_agent(
    model=local_llm,
    tools=[download_viirs_snpp_peru, fires_clean,
           download_ctry_shp, intersect_and_collapse_fires_peru,
           folium_choropleth_peru],
    system_prompt=system_prompt,
    skills=[str(SCRIPT_DIR)]  # carpeta con SKILL.md
)
```

Resultado de la demo

Query: descargar y analizar los incendios forestales de Perú en 2024



Descarga y limpieza

99,932 focos de calor VIIRS-SNPP descargados y depurados para 2024



Agregación por departamento

Ucayali (18,532), San Martín (12,848) y Loreto (11,761) concentran más incendios



Mapa interactivo

`folium_choropleth_peru()` genera un HTML con el conteo de incendios por departamento

Fundamentos para el desarrollo de Agentes de IA

MSc. Jesus Gastañaduy